



Université Abdelmalek Essaadi
Faculté Des Sciences Et Techniques Al Hoceima (FSTH)



Projet académique :

Poussette intelligente

Présenté à FSTH

Master sciences et techniques en Systèmes Embarqués et
Robotiques

Réalisé Par :

Prof :

▪ *Hamid El Moumen*

1-Bouzakoura Oussama

2-Fattachi Imane

3-Hassan Abdoulaye acyl

4-Hamza Ait Taleb

Année Universitaire 2025-2026

RÉSUMÉ DU PROJET

Face aux contraintes physiques des poussettes conventionnelles lors de déplacements longs ou sur des terrains en pente, ce projet propose la conception d'une poussette autonome et intelligente dédiée à la puériculture.

Intégrant des capteurs infrarouges, ultrasoniques et un capteur inertiel (MPU6050) pilotés par un microcontrôleur, ce système embarqué est capable de suivre automatiquement l'utilisateur, de détecter et d'éviter les obstacles, et d'ajuster sa vitesse en fonction de l'inclinaison du terrain.

Cette solution innovante allie robotique et sécurité pour offrir une mobilité assistée, stable et confortable aux parents.

SOMMAIRE

RÉSUMÉ DU PROJET	2
SOMMAIRE	3
Introduction.....	6
Contexte du projet	7
Problématique et objectifs	7
I. Problématique.....	7
II. Objectifs.....	7
1. Objectif Général	7
2. Objectif Spécifique	7
Chapitre 1 : Étude théorique et présentation générale du système.....	9
I. Présentation générale du projet	9
II. Description du fonctionnement Global.....	9
III. Cahier des Charges fonctionnel	10
Fonction Principale	10
Fonction secondaire	10
Contraintes du projet	10
IV. Description de sous systèmes	11
Système de Suivi de l'Utilisateur	11
Système d'Évitement d'Obstacles	11
Système de Gestion de pente	11
Système de commande de moteurs.....	11
Chapitre 2 : Conception de système	12
I. Introduction.....	12
II. Architecture générale du système	12
III. Conception du Système d'Évitement d'Obstacles et d'Alerte	12
III. Conception du Système de Détection de Pente	13
IV. Conception du système de commande des moteurs	13
V. Conception de détection de présence de parents	13
1. Temporisation de sécurité	14
VI. Conception du Système de Commande de moteur par Télécommande.....	14
VII. Protocole I ² C	15
1. Définition	15
2. Lecture des données d'accélération	16
3. Principe de fonctionnement	16

VIII.	Protocole UART	16
1.	Définition	16
2.	Principe de fonctionnement	16
IX.	Choix des Composants	17
1.	Microcontrôleur PIC16F887	17
1.1.	Définition.....	17
1.2.	Rôle dans le projet.....	17
1.3.	Pourquoi le choix du PIC ?.....	18
1.4.	Architecture et Caractéristiques Techniques	18
1.5.	Le Brochage (Pinout)	19
1.6.	L'Horloge (Clock)	19
1.7.	Les Périphériques Internes.....	20
2.	Châssis avec 4 moteurs et 4 roues.....	21
3.	Capteur Gyroscope et Accéléromètre MPU6050	21
4.	Émetteur infrarouge KY-005	21
5.	Récepteur infrarouge KY-022.....	21
6.	Interrupteur KCD11-101	21
7.	Capteur Ultrason HC-SR04	21
8.	Contrôleur / Shield L293D.....	21
9.	Alimentation ajustable LM2596 3A	22
10.	Batterie Li-ion 18650 3,7V	22
11.	Arduino Uno	22
12.	Module HC-12 (Radio)	22
13.	Boutons Poussoirs	22
14.	Buzzer	22
15.	LED	22
X.	Liste des matériels.....	23
XI.	Algorithme général du système	23
Chapitre 3 : Simulation		26
I.	Présentation du logiciel Proteus ISIS.....	26
1.	Description d'interface du logiciel Proteus ISIS	26
1.1.	Page d'accueil de Proteus Design Suite.....	26
8.1.	Création d'un nouveau projet sous Proteus	28
8.2.	Zone de travail (Workspace)	29
II.	Présentation du schéma.....	30
III.	Description du fonctionnement général.....	30

IV.	Choix des composants de simulation.....	31
V.	Fonctionnement global du système	31
VI.	Résultats de simulation.....	31
Chapitre 4 : Réalisation et Test de validation.....		33
I.	Introduction.....	33
II.	Schéma électronique du montage	33
III.	Développement du logiciel	34
1.	MPLAB X IDE.....	34
1.1.	Création d'un projet PIC16F887	34
1.2.	Programmation en Assembleur (ASM).....	34
1.3.	Génération du fichier HEX.....	34
IV.	Tests et Validation.....	35
1.	Test du Système d'évitement d'obstacle.....	35
2.	Test du Système de Détection de Pente.....	35
3.	Test de système de détection de présence de parents	36
4.	Test du Système de Commande de moteur par Télécommande	36
V.	Analyse de performance	36
VI.	Limites du système et Difficultés rencontrés :.....	37
1.	Limites du système.....	37
2.	Difficultés rencontrées.....	37
Conclusion général		39
Annexes :		40

Introduction

L'évolution rapide des technologies embarquées, de la robotique et des systèmes intelligents a permis l'apparition de nouvelles solutions visant à améliorer le confort, la sécurité et l'autonomie dans la vie quotidienne. Aujourd'hui, les capteurs intelligents, les microcontrôleurs et les systèmes de commande sont largement utilisés dans de nombreux domaines tels que l'industrie, la santé, les transports et les équipements domestiques.

Dans le domaine de la puériculture, les poussettes conventionnelles demeurent le principal moyen de transport des jeunes enfants. Cependant, leur utilisation peut parfois devenir contraignante pour les parents, notamment lors des déplacements sur de longues distances, dans des environnements encombrés ou sur des terrains présentant des pentes importantes. Ces contraintes peuvent entraîner une fatigue supplémentaire et réduire le confort d'utilisation.

Afin de répondre à ces besoins, notre projet consiste à concevoir et réaliser une poussette autonome intelligente capable d'accompagner automatiquement son utilisateur tout en garantissant la sécurité de l'enfant. Cette poussette est équipée de plusieurs capteurs permettant de détecter la présence de l'utilisateur, d'éviter les obstacles et d'adapter son comportement aux conditions de déplacement rencontrées.

Le système proposé repose sur l'utilisation de capteurs infrarouges, de capteurs ultrasoniques et d'un capteur inertiel MPU6050 associé à un microcontrôleur chargé du traitement des données et de la commande des moteurs. Grâce à cette architecture, la poussette est capable d'ajuster automatiquement sa vitesse en fonction de l'inclinaison du terrain afin de maintenir un déplacement stable et sécurisé.

Ce projet s'inscrit dans une démarche d'innovation technologique visant à développer une solution intelligente capable d'améliorer l'expérience des utilisateurs tout en démontrant l'apport des systèmes embarqués dans les applications de mobilité assistée.

Contexte du projet

De nos jours, les déplacements avec de jeunes enfants font partie intégrante du quotidien des parents. La poussette constitue le moyen de transport le plus utilisé pour assurer le confort et la mobilité des nourrissons et des jeunes enfants. Cependant, les poussettes traditionnelles nécessitent une intervention permanente de l'utilisateur pour assurer leur déplacement et leur guidage.

Dans les environnements urbains, les parents sont souvent confrontés à plusieurs difficultés telles que la présence d'obstacles, l'affluence des piétons, les changements de direction fréquents ainsi que les routes présentant des montées ou des descentes. Ces situations peuvent rendre l'utilisation d'une poussette classique moins confortable et nécessiter un effort physique important, notamment lors des trajets de longue distance.

Par ailleurs, la sécurité de l'enfant demeure une préoccupation majeure. Une mauvaise maîtrise de la poussette dans une pente ou à proximité d'un obstacle peut présenter un risque pour l'enfant et pour son accompagnateur.

Face à ces contraintes, l'intégration de technologies embarquées dans une poussette apparaît comme une solution innovante permettant d'améliorer à la fois le confort, l'assistance et la sécurité des déplacements.

Problématique et objectifs

I. Problématique

Cependant, comment concevoir une poussette capable :

- ✓ D'accompagner automatiquement son utilisateur
- ✓ D'éviter les obstacles
- ✓ D'assurer la sécurité de l'enfant
- ✓ D'adapter automatiquement sa vitesse selon la pente

II. Objectifs

1. Objectif Général

L'objectif principal de ce projet est de concevoir et réaliser une poussette autonome intelligente reposant sur une architecture embarquée permettant d'assister l'utilisateur lors de ses déplacements tout en assurant la sécurité de l'enfant.

2. Objectif Spécifique

Pour atteindre cet objectif général, plusieurs objectifs spécifiques ont été définis :

- Concevoir un système de suivi automatique permettant à la poussette de se déplacer devant son utilisateur.
- Mettre en place un système de détection et d'évitement d'obstacles afin d'assurer un déplacement sécurisé.
- Intégrer un capteur inertiel pour mesurer en temps réel l'inclinaison de terrain.
- Développer un algorithme de commande permettant d'adapter automatiquement la vitesse de moteur selon la pente détectée.

- Augmenter la vitesse de déplacement lors d'une montée afin de compenser l'effort supplémentaire nécessaire.
- Réduire automatiquement la vitesse lors d'une descente afin d'améliorer la stabilité et la sécurité.
- Maintenir une vitesse constante lorsque le terrain est plat.
- Assurer une communication efficace entre les capteurs, le microcontrôleur et les actionnaires.
- Valider expérimentalement le bon fonctionnement du système à travers différents scénarios de test.

Chapitre 1 : Étude théorique et présentation générale du système

I. Présentation générale du projet

Les avancées technologiques dans le domaine des systèmes embarqués et de la robotique permettent aujourd'hui de développer des équipements intelligents capables d'assister les utilisateurs dans leurs tâches quotidiennes. Dans ce contexte, notre projet consiste à concevoir une poussette autonome intelligente destinée à faciliter le déplacement des parents tout en garantissant la sécurité de l'enfant.

Contrairement aux poussettes traditionnelles qui nécessitent une intervention permanente de l'utilisateur, la poussette proposée est capable de suivre automatiquement son propriétaire grâce à un système de détection embarqué. Elle est également capable de détecter les obstacles présents sur son chemin et d'adapter son comportement afin d'éviter les collisions.

L'une des particularités de ce projet réside dans l'intégration d'un système de gestion de pente basé sur le capteur MPU6050. Ce capteur mesure en temps réel l'inclinaison de la poussette et transmet les informations au microcontrôleur. En fonction de la pente détectée, la vitesse des moteurs est automatiquement ajustée par modulation PWM :

- Lorsque la poussette monte une pente, la vitesse augmente afin de compenser l'effort supplémentaire nécessaire au déplacement.
- Lorsque la poussette descend une pente, la vitesse diminue afin d'améliorer la stabilité et la sécurité.
- Sur un terrain plat, la vitesse reste constante.

L'objectif principal de cette approche est d'offrir un déplacement plus confortable, plus sécurisé et plus autonome pour les parents et leurs enfants.

II. Description du fonctionnement Global

Le fonctionnement de la poussette repose sur plusieurs sous-systèmes travaillant ensemble. Tout d'abord, les capteurs infrarouges permettent de détecter la position de l'utilisateur afin que la poussette puisse le suivre automatiquement.

Ensuite, les capteurs ultrasoniques surveillent en permanence l'environnement proche afin de détecter les obstacles susceptibles de gêner le déplacement. Lorsqu'un obstacle est détecté à une distance critique, le système peut ralentir ou arrêter la poussette afin d'éviter toute collision.

Par ailleurs, le capteur MPU6050 mesure l'orientation de la poussette par rapport au sol. Les données recueillies sont traitées par le microcontrôleur qui détermine si le terrain est plat, en montée ou en descente.

Enfin, le microcontrôleur commande les moteurs à courant continu par l'intermédiaire d'un signal PWM. Cette commande permet de modifier la vitesse de rotation des moteurs afin d'adapter automatiquement le comportement de la poussette aux conditions réelles de déplacement.

III. Cahier des Charges fonctionnel

Le cahier des charges regroupe l'ensemble des besoins auxquels le système doit répondre.

Fonction Principale

La fonction principale de la poussette autonome est d'assurer le transport sécurisé de l'enfant tout en suivant automatiquement son utilisateur.

Fonction secondaire

Afin de réaliser sa fonction principale, la poussette doit assurer plusieurs fonctions secondaires :

- Détecter et suivre l'utilisateur
- Détecter les obstacles présents sur le parcours
- Éviter les collisions
- Adapter sa vitesse selon la pente du terrain
- Assurer la sécurité de l'enfant
- Garantir un déplacement stable et confortable
- Optimiser la consommation d'énergie

Contraintes du projet

Le système doit également respecter certaines contraintes :

a. Contraintes Techniques

- Utilisation de composants électroniques fiables
- Réactivité suffisante face aux obstacles
- Fonctionnement en temps réel

b. Contraintes de sécurité

- Arrêt rapide en présence de danger
- Stabilité de la poussette lors des déplacements
- Protection de l'enfant contre les mouvements brusques

c. Contraintes économiques

- Coût de réalisation raisonnable
- Facilité d'entretien
- Disponibilité des composants sur le marché

d. Analyse fonctionnelle du système

L'analyse fonctionnelle consiste à identifier les interactions entre la poussette et son environnement :

- Entrée du système

Les informations reçues par le système sont :

- ✓ Position de l'utilisateur
- ✓ Distance des obstacles
- ✓ Angle d'inclinaison du terrain
- ✓ État de la batterie

- Traitement :

Le microcontrôleur analyse les informations reçues et prend les décisions nécessaires concernant :

- ✓ La direction de déplacement
- ✓ La vitesse
- ✓ L'arrêt éventuel du système

- Sortie du système :

Les sorties du système sont :

- ✓ Commande des moteurs
- ✓ Variation de vitesse
- ✓ Arrêt de sécurité
- ✓ Déplacement de la poussette
- ✓ Alerte via une LED

IV. Description de sous systèmes

Système de Suivi de l'Utilisateur

Ce sous-système permet à la poussette de maintenir une distance raisonnable avec son utilisateur. Les capteurs installés sur la poussette détectent la position de la personne et transmettent ces informations au microcontrôleur qui ajuste automatiquement la direction et la vitesse des moteurs.

Système d'Évitement d'Obstacles

Le système utilise des capteurs ultrasoniques afin de mesurer la distance séparant la poussette des objets environnants. Lorsqu'un obstacle est détecté, le système peut ralentir ou arrêter la poussette afin d'éviter tout risque de collision.

Système de Gestion de pente

Le capteur MPU6050 mesure en permanence l'inclinaison de la poussette, lorsque l'angle mesuré indique une montée, le microcontrôleur augmente le rapport cyclique du signal PWM afin de fournir davantage de puissance aux moteurs.

Lorsque l'angle indique une descente, le rapport cyclique est réduit afin de limiter la vitesse, Si l'angle reste proche de zéro, le système maintient la vitesse nominale programmée.

Système de commande de moteurs

La commande des moteurs est réalisée par le microcontrôleur à travers des signaux PWM. Cette technique permet de contrôler précisément la vitesse de rotation des moteurs tout en optimisant la consommation énergétique.

Dans ce chapitre, nous avons présenté le contexte général du projet ainsi que les principales fonctions attendues de la poussette autonome. L'analyse fonctionnelle a permis d'identifier les besoins du système, les contraintes à respecter ainsi que les différents sous-systèmes nécessaires à son fonctionnement. Cette étude constitue la base de la phase de conception qui sera développée dans le chapitre suivant.

Chapitre 2 : Conception de système

I. Introduction

Après avoir identifié les besoins fonctionnels du système dans le chapitre précédent, il est nécessaire de concevoir une architecture capable de répondre aux différentes exigences du projet. Cette étape permet de définir les composants matériels et logiciels nécessaires ainsi que leurs interactions.

La poussette autonome développée dans le cadre de ce projet repose sur une architecture embarquée associant plusieurs capteurs, un microcontrôleur et des actionneurs. L'ensemble de ces éléments travaille en temps réel afin d'assurer le suivi automatique de l'utilisateur, l'évitement des obstacles et l'adaptation de la vitesse en fonction de l'inclinaison du terrain.

II. Architecture générale du système

Le système est composé de quatre parties principales :

- Le système de suivi de l'utilisateur
- Le système d'évitement d'obstacles
- Système de détection et de présence du parent
- Système d'alerte et de sécurité
- Le système de détection de pente
- Le système de commande des moteurs

Les informations provenant des différents capteurs sont transmises au microcontrôleur qui les traite puis génère les signaux de commande nécessaires aux moteurs.

III. Conception du Système d'Évitement d'Obstacles et d'Alerte

Pour assurer la sécurité de l'enfant, la poussette est équipée d'un capteur ultrasonique chargé de surveiller en permanence l'espace situé devant elle.

Le capteur mesure continuellement la distance entre la poussette et les objets présents sur son parcours.

Lorsque la distance mesurée devient inférieure à une valeur de sécurité prédéfinie :

- La poussette s'arrête immédiatement ;
- Un buzzer est activé. Cette stratégie permet de réduire considérablement les risques de collision.

Le buzzer produit un signal sonore permettant d'avertir l'utilisateur de la présence d'un obstacle. Cette alerte sonore, améliore considérablement la sécurité du système et permet une réaction rapide de l'utilisateur.

Une fois l'obstacle supprimé ou contourné, le système peut reprendre son fonctionnement.

III. Conception du Système de Détection de Pente

L'une des fonctionnalités innovantes de ce projet consiste à adapter automatiquement la vitesse de la poussette en fonction de la pente du terrain.

secondaires :

Pour cela, nous utilisons le **module MPU6050** qui intègre :

- Un accéléromètre ;
- Un gyroscope.

Ces deux capteurs permettent de déterminer l'inclinaison de la poussette par rapport au sol.

Le microcontrôleur lit régulièrement les données envoyées par le MPU6050 et calcule l'angle d'inclinaison.

Trois situations peuvent être rencontrées :

Cas 1 : Terrain plat

Lorsque l'angle mesuré est proche de zéro :

- La vitesse programmée est maintenue

Cas 2 : Montée

Lorsque l'angle est supérieur à une valeur seuil :

- Le rapport cyclique du PWM est augmenté
- La vitesse des moteurs augmente
- La LED vert s'allume

Cas 3 : Descente

Lorsque l'angle devient inférieur au seuil négatif :

- Le rapport cyclique du PWM est réduit
- La vitesse diminue afin d'améliorer la stabilité du déplacement
- La LED rouge s'allume

IV. Conception du système de commande des moteurs

Les moteurs à courant continu assurent le déplacement de la poussette. la variation de leur vitesse est obtenue grâce à la technique PWM (Pulse Width Modulation). le principe consiste à faire varier le rapport cyclique du signal de commande appliqué aux moteurs.

Lorsque le rapport cyclique augmente :

- La tension moyenne appliquée au moteur augmente ;
- La vitesse de rotation augmente.

Lorsque le rapport cyclique diminue :

- La tension moyenne diminue.
- La vitesse diminue.

Cette technique offre une commande simple, efficace.

V. Conception de détection de présence de parents

La sécurité de l'enfant constitue l'une des principales priorités de ce projet. Pour cette raison, un système spécifique a été développé afin de vérifier en permanence que le parent se trouve à proximité de la poussette.

Ce système repose sur un émetteur infrarouge KY-005 et trois capteurs récepteurs infrarouges. L'émetteur KY-005 est intégré à la télécommande portée par le parent. Celui-ci émet continuellement un signal infrarouge modulé à une fréquence de 38 kHz.

Du côté de la poussette, trois récepteurs infrarouges sont installés :

- Un récepteur à gauche
- Un récepteur au centre

- Un récepteur à droite

Cette disposition permet d'élargir la zone de détection et d'assurer la réception du signal quelle que soit la position du parent autour de la poussette.

Le principe de fonctionnement est le suivant :

Lorsque le parent se trouve à proximité de la poussette, le signal émis par le KY-005 est capté par au moins l'un des trois récepteurs infrarouges. le microcontrôleur considère alors que le parent est présent et autorise le fonctionnement normal du système.

Dans cette situation :

- La poussette peut continuer à se déplacer
- Les commandes de la télécommande sont acceptées
- La LED verte reste allumée

Cependant, lorsque les trois récepteurs infrarouges cessent de détecter le signal émis par le parent, le microcontrôleur considère qu'une perte de présence est possible.

Afin d'éviter un arrêt intempestif provoqué par une perturbation momentanée, un temporisateur de sécurité est lancé.

1. Temporisation de sécurité

Dès que la détection disparaît :

- Un compteur de 5 secondes démarre
- La poussette continue temporairement son fonctionnement

Si le signal infrarouge est de nouveau détecté avant la fin de cette période :

- ✓ Le compteur est remis à zéro
- ✓ Le fonctionnement normal reprend

En revanche, si aucune détection n'est réalisée pendant plus de 5 secondes :

- ✓ Le microcontrôleur considère que le parent n'est plus présent
- ✓ Les moteurs sont immédiatement arrêtés
- ✓ La LED rouge est activée
- ✓ Toutes les commandes de déplacement sont bloquées

Cette stratégie améliore considérablement la sécurité du système en empêchant tout déplacement incontrôlé de la poussette lorsque l'utilisateur s'éloigne ou perd le contrôle de la télécommande.

VI. Conception du Système de Commande de moteur par Télécommande

Afin de permettre à l'utilisateur de contrôler le déplacement de la poussette à distance, nous avons conçu un système de communication sans fil basé sur deux modules HC-12. Ces modules assurent la transmission et la réception des commandes entre la télécommande et la poussette. Le premier module HC-12 est connecté à une carte Arduino qui joue le rôle d'émetteur. le second module HC-12 est connecté au microcontrôleur PIC16F877A qui joue le rôle de récepteur.

La télécommande est constituée de deux boutons poussoirs :

- Un bouton de direction gauche
- Un bouton de direction droite

Lorsque l'utilisateur appuie sur l'un des boutons, l'Arduino génère un message correspondant à la commande demandée. Ce message est ensuite transmis par le module HC-12 émetteur.

Le module HC-12 récepteur installé sur la poussette reçoit le message puis le transmet au microcontrôleur PIC pour traitement.

Le fonctionnement adopté est le suivant :

➤ *Commande vers la gauche*

Lorsque le bouton gauche est actionné

- L'Arduino envoie une commande « Gauche »
- Le HC-12 transmet le message
- Le PIC reçoit l'information
- Le moteur est commandé de manière à orienter la poussette vers la gauche

➤ *Commande vers la droite*

Lorsque le bouton droit est actionné

- L'Arduino envoie une commande « Droite »
- Le HC-12 transmet la commande
- Le PIC décode le message
- La poussette effectue une déviation vers la droite

➤ *Commande d'arrêt*

Lorsque les deux boutons sont appuyés simultanément

- L'Arduino génère une commande d'arrêt
- Le message est envoyé par le HC-12
- Le PIC identifie cette commande
- Tous les moteurs sont immédiatement arrêtés

Cette fonction permet à l'utilisateur d'interrompre rapidement le déplacement de la poussette en cas de danger ou de situation imprévue.

L'utilisation des modules HC-12 présente plusieurs avantages

- Communication sans fil sur plusieurs dizaines de mètres
- Simplicité de mise en œuvre
- Faible consommation énergétique
- Bonne fiabilité de transmission

VII. Protocole I²C

1. Définition

Le protocole I²C (Inter-Integrated Circuit) est un protocole de communication série synchrone utilisant seulement deux lignes :

- **SDA** (Serial Data) : transfert des données
- **SCL** (Serial Clock) : signal d'horloge

L'I²C fonctionne selon le principe Maître-Esclave. Le maître contrôle l'horloge et démarre la communication, tandis que l'esclave répond à son adresse. Plusieurs périphériques peuvent partager le même bus I²C.

Utilisation avec le **PIC16F887** et le **MPU6050**

Dans ce montage :

- Le PIC16F887 est configuré en maître I²C grâce au module MSSP.
- Le MPU6050 est un esclave I²C dont l'adresse est généralement 0x68.

Le PIC16F887 communique avec le MPU6050 via les broches :

- RC3 → SCL
- RC4 → SDA

Le module MSSP du PIC permet de gérer automatiquement les opérations I²C (START, STOP, ACK, transmission et réception).

2. Lecture des données d'accélération

Le MPU6050 stocke les mesures des axes X, Y et Z dans les registres suivants :

Axe	Registre High	Registre Low
X	0x3B	0x3C
Y	0x3D	0x3E
Z	0x3F	0x40

Ces registres contiennent les valeurs brutes de l'accéléromètre sur 16 bits.

3. Principe de fonctionnement

1. Initialiser le module I²C du PIC16F887
2. Envoyer la condition *START*
3. Envoyer l'adresse du MPU6050 (0x68)
4. Sélectionner le registre de départ (0x3B)
5. Lire les 6 octets correspondant aux axes X, Y et Z
6. Combiner les octets High et Low
7. Envoyer la condition *STOP*

VIII. Protocole UART

1. Définition

Le protocole **UART** (Universal Asynchronous Receiver Transmitter) est un protocole de communication série asynchrone permettant l'échange de données entre deux dispositifs à l'aide de deux lignes : TX (Transmission) et RX (Réception). Contrairement à l'I²C, il ne nécessite pas de signal d'horloge. Les deux équipements doivent simplement utiliser le même débit de transmission (baud rate), par exemple 9600 bps.

Utilisation avec le **PIC16F887** et le **HC-12**

Dans cette application :

- Le PIC16F887 utilise son module EUSART/UART intégré
- Le HC-12 est un module radio 433 MHz qui transmet les données UART sans fil
- Un Arduino équipée d'un second HC-12 envoie l'état d'un bouton poussoir
- Le HC-12 connecté au PIC reçoit cette information et la transmet au PIC via UART

Connexions

PIC16F887	HC-12
RC6/TX	RXD
RC7/RX	TXD
VCC 5V	VCC
GND	GND

Les broches UART du PIC16F887 sont **RC6 (TX)** et **RC7 (RX)**.

2. Principe de fonctionnement

1. L'Arduino lit l'état d'un bouton.
2. Si le bouton est appuyé, elle envoie par exemple le caractère '1'.
3. Le HC-12 transmet cette donnée par radio.
4. Le HC-12 récepteur reçoit la donnée.
5. Le PIC16F887 lit le caractère via UART.
6. Le PIC exécute une action (changer les directions des moteurs).

IX. Choix des Composants

Pour concrétiser notre étude théorique, nous avons sélectionné des composants

1. Microcontrôleur PIC16F887

1.1. Définition

Les **microcontrôleurs PIC** (ou PICmicro dans la terminologie du fabricant) forment une famille de microcontrôleurs de la société Microchip. Ces microcontrôleurs sont dérivés du PIC1650 développé à l'origine par la division microélectronique de General Instrument.

Le nom PIC n'est pas officiellement un acronyme, bien que la traduction en « *Peripheral Interface Controller* » (« contrôleur d'interface périphérique ») soit généralement admise. Cependant, à l'époque du développement du PIC1650 par General Instrument, PIC était un acronyme de « *Programmable Intelligent Computer* » ou « *Programmable Integrated Circuit* ».

Un microcontrôleur PIC est une unité de traitement et d'exécution de l'information à laquelle on a ajouté des périphériques internes permettant de réaliser des montages sans nécessiter l'ajout de composants annexes. Un microcontrôleur PIC peut donc fonctionner de façon autonome après programmation.

Les PIC intègrent une mémoire programme non volatile (FLASH), une mémoire de données volatile, une mémoire de donnée non volatile (EEPROM), des ports d'entrée-sortie (numériques, analogiques, MLI, UART, bus I²C, Timers, SPI, etc.), et même une horloge, bien que des bases de temps externes puissent être employées. Certains modèles disposent de ports et unités de traitement de l'USB et Ethernet.

Les PIC de la famille 16C ou 16F sont des composants de milieu de gamme. C'est la famille la plus riche en termes de dérivés.

La Famille 16F dispose dorénavant de trois sous-familles :

- La sous-famille avec le cœur Baseline : instructions sur 12 bits (PIC16Fxxx) ;
- La sous-famille avec le cœur Middle-Range : instructions sur 14 bits (PIC16Fxxx) ;
- La sous-famille avec le cœur Enhanced : instructions sur 14 bits (PIC16F1xxx et PIC16F1xxxx) avec 49 instructions.

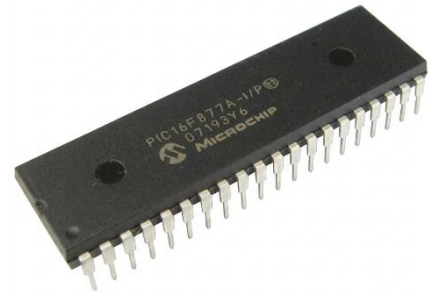
PIC16F1xxx : il s'agit d'une nouvelle famille (2009) créée spécifiquement pour permettre l'extension de la mémoire FLASH, de la mémoire RAM et l'ajout de périphériques, tout en gardant la compatibilité avec les cœurs baseline et middle-range. L'ajout d'une quinzaine d'instructions orientée pour les compilateurs C permettent de diminuer de façon significative la taille du code généré (jusqu'à 40 % de moins que le cœur PIC16 Middle-range).

1.2. Rôle dans le projet

Dans l'architecture globale de notre projet, le microcontrôleur PIC assume le rôle critique de cerveau du système embarqué. Son rôle principal est de gérer la chaîne d'information et la chaîne d'énergie à travers un cycle continu et en temps réel :

Acquérir → Traiter → Communiquer → Agir.

Ainsi, il acquiert les données des capteurs (ultrason, capteurs IR, MPU6050...), traite ces informations de manière algorithmique pour prendre des décisions logiques, communique avec les modules internes et externes (HC-12), puis agit sur son environnement en transmettant des



signaux de commande aux actionneurs (moteurs via le L293D, LEDs, buzzer). Ce cycle, répété en boucle à haute fréquence, est la clé de voûte de l'autonomie et de la réactivité du robot.

1.3. Pourquoi le choix du PIC ?

- **Robustesse industrielle :**

Les microcontrôleurs PIC sont largement utilisés dans les systèmes industriels grâce à leur fiabilité, leur stabilité et leur capacité à fonctionner dans des environnements exigeants. Certains modèles sont même certifiés pour des applications industrielles et automobiles.

- **Rapport performance/coût :**

Les PIC offrent des performances suffisantes pour les systèmes embarqués tout en conservant un coût réduit, ce qui permet de limiter le coût global du produit.

- **Apprentissage académique :**

Les microcontrôleurs PIC sont largement utilisés dans l'enseignement de l'électronique et des systèmes embarqués. Leur architecture simple, leur documentation abondante et les nombreux outils de développement disponibles facilitent l'apprentissage et la réalisation de projets pédagogiques.

1.4. Architecture et Caractéristiques Techniques

- **Architecture Harvard & RISC**

- ✓ *Architecture Harvard*

Le PIC16F887 utilise une architecture Harvard, ce qui signifie que la mémoire programme et la mémoire de données sont séparées. Cette organisation permet au microcontrôleur d'accéder simultanément aux instructions et aux données, améliorant ainsi la vitesse d'exécution.

- ✓ *Architecture RISC*

Le PIC16F887 est basé sur une architecture RISC (Reduced Instruction Set Computer) caractérisée par :

Un jeu réduit de seulement 35 instructions.

La plupart des instructions s'exécutent en un seul cycle machine.

Une programmation plus simple et une exécution rapide.

La Mémoire

- **Mémoire Flash (Programme)**

Capacité : 14 Ko (8K mots de 14 bits).

Utilisée pour stocker le programme (code source compilé).

Mémoire non volatile : les données restent enregistrées même après la coupure de l'alimentation.

Peut-être reprogrammée plusieurs fois.

- **Mémoire RAM**

Capacité : 368 octets.

Utilisée pour stocker les variables et les données temporaires pendant l'exécution du programme.

Mémoire volatile : son contenu est perdu lorsque l'alimentation est coupée.

- **Mémoire EEPROM**

Capacité : 256 octets.

Utilisée pour conserver des données importantes (paramètres, configurations, mesures, etc.).

Mémoire non volatile : les données sont conservées même sans alimentation.

Peut être modifiée directement par le programme durant le fonctionnement du microcontrôleur.

1.5. Le Brochage (Pinout)

Le PIC16F887 possède 40 broches réparties en 5 ports d'entrées/sorties : PORTA, PORTB, PORTC, PORTD et PORTE.

Port	Broches
PORTA	RA0 à RA7
PORTB	RB0 à RB7
PORTC	RC0 à RC7
PORTD	RD0 à RD7
PORTE	RE0 à RE3

Broches principales

- MCLR/VPP (Pin 1) : Reset et programmation
- VDD (Pins 11 et 32) : Alimentation (+5V)
- VSS (Pins 12 et 31) : Masse (GND)
- OSC1 (RA7) et OSC2 (RA6) : Connexion du quartz externe
- RB0/INT : Interruption externe
- RC6/TX et RC7/RX : Communication série UART
- RC3/SCL et RC4/SDA : Communication I²C
- RC3/SCK, RC4/SDI, RC5/SDO : Communication SPI

1.6. L'Horloge (Clock)

Le PIC16F887 possède un module d'horloge flexible permettant d'utiliser une source d'horloge interne ou externe.

➤ Horloge interne

Oscillateur interne HFINTOSC : jusqu'à 8 MHz.

Oscillateur interne LFINTOSC : 31 kHz.

Fréquences sélectionnables par logiciel :

- 8 MHz
- 4 MHz
- 2 MHz
- 1 MHz
- 500 kHz
- 250 kHz
- 125 kHz
- 31 kHz

➤ Horloge externe

Le PIC16F887 peut également utiliser :

- Un quartz (Crystal)
- Un résonateur céramique
- Une horloge externe
- Un circuit RC externe

Avec une fréquence pouvant atteindre 20 MHz.

1.7. Les Périphériques Internes

Le PIC16F887 intègre plusieurs périphériques matériels permettant de réaliser des applications embarquées sans composants supplémentaires.

Convertisseur Analogique-Numérique (ADC)

- ADC 10 bits.
- Jusqu'à 14 entrées analogiques.
- Conversion des signaux analogiques en données numériques.

Timers

- Timer0 : 8 bits.
- Timer1 : 16 bits.
- Timer2 : 8 bits.
- Utilisés pour les temporisations, mesures de temps et génération de signaux.

Modules CCP

- Modules CCP (Capture/Compare/PWM).
- Génération de signaux PWM.
- Mesure et comparaison de signaux.

Communication USART

- Communication série asynchrone (UART).
- Transmission et réception de données.

Communication MSSP

Support des protocoles :

- I²C
- SPI

Compareurs Analogiques

- 2 compareurs analogiques intégrés.
- Comparaison de tensions analogiques.

Watchdog Timer (WDT)

- Redémarrage automatique du microcontrôleur en cas de blocage.

Brown-Out Reset (BOR)

- Protection contre les chutes de tension.

Power-On Reset (POR)

- Réinitialisation automatique à la mise sous tension.

Interruption Externe et Interne

- Interruptions sur événements matériels.
- Gestion rapide des événements.

Oscillateur Interne

- Oscillateur interne jusqu'à 8 MHz.
- Possibilité d'utiliser un quartz externe jusqu'à 20 MHz.

EEPROM de Données

- 256 octets de mémoire EEPROM intégrée.
- Stockage permanent des données

2. Châssis avec 4 moteurs et 4 roues

Constitue la structure mécanique de la poussette et assure sa mobilité globale.

La configuration à 4 roues motrices (4WD) offre une excellente stabilité et une meilleure traction pour transporter une charge en toute sécurité.



3. Capteur Gyroscope et Accéléromètre MPU6050

Mesure l'inclinaison (tangage/roulis) de la poussette et détecte les accélérations soudaines ou les risques de basculement.

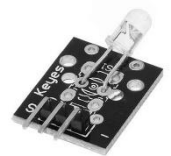
Indispensable pour la sécurité : il permet de détecter si la poussette est sur une pente dangereuse ou si elle est en train de se renverser ou elle est très lent afin de régler la vitesse pour qu'elle soit similaire à la vitesse des parents.



4. Émetteur infrarouge KY-005

Émet un signal lumineux infrarouge modulé (souvent à 38 kHz) pour concevoir un système de suivi ou de barrière.

Composant très économique, simple à programmer en Assembleur et idéal pour l'interfaçage directionnel à courte portée.



5. Récepteur infrarouge KY-022

Capte les signaux infrarouges émis par Émetteur infrarouge KY-005. La configuration à 3 récepteurs permet de faire de la triangulation ou de détecter la direction d'une source. Dans notre projet c'est la détecte d'existence des parents et les suivis automatiquement).

Haute sensibilité aux signaux modulés, ce qui immunise le système contre les perturbations de la lumière ambiante.



6. Interrupteur KCD11-101

Permet de couper ou d'établir l'alimentation générale du système (ON/OFF).

Assure la sécurité matérielle en permettant un arrêt d'urgence électrique et évite la décharge des batteries au repos.



7. Capteur Ultrason HC-SR04

Mesure la distance entre la poussette et les obstacles potentiels situés à l'avant.

Permet d'implémenter l'algorithme d'évitement de collision ou de freinage automatique grâce à sa précision de mesure (2 cm à 400 cm).



8. Contrôleur / Shield L293D

Reçoit les signaux de commande logiques du PIC et fournit la puissance nécessaire (courant/tension) aux moteurs DC.

Intègre un double pont en H capable de piloter le sens de rotation et la vitesse (via PWM) de plusieurs moteurs, tout en protégeant le PIC des retours de courant.



9. Alimentation ajustable LM2596 3A

Régule et abaisse la tension de la batterie à une valeur stable de 5V pour alimenter proprement le PIC et les capteurs.

C'est un régulateur à découpage (Buck) à fort rendement ; il ne chauffe presque pas par rapport à un régulateur linéaire (type LM7805) et délivre jusqu'à 3A.



10. Batterie Li-ion 18650 3,7V

Une source d'énergie principale du système. Associées en série, elles fournissent la tension et l'autonomie nécessaires aux moteurs 12 V et les 5 V du PIC et des capteurs en utilisant l'abaisseur.

Offrent une excellente densité énergétique, une tension stable et sont rechargeables, ce qui est idéal pour un système embarqué mobile.



11. Arduino Uno

Le Cerveau de la manette : Il lit en continu l'état des boutons et traduit l'appui en ordres codés à envoyer aux émetteurs.

Idéal pour le prototypage rapide. Il consomme très peu d'énergie sur pile et dispose de toutes les bibliothèques prêtes pour l'infrarouge et le HC-12.



12. Module HC-12 (Radio)

Liaison longue portée et robuste : Il envoie les commandes de direction (gauche/droite) par ondes radio (433 MHz) vers le HC-12 du robot.

Portée énorme (jusqu'à 1 km). Contrairement à l'infrarouge, les ondes radio traversent les obstacles légers (vêtements, sac) et fonctionnent même si la télécommande est dans la poche de l'utilisateur.



13. Boutons Poussoirs

Commandes manuelles de direction : Permettent à l'utilisateur de forcer la poussette à tourner à gauche ou à droite si nécessaire.

Offre une sécurité et un contrôle manuel total à l'utilisateur pour corriger la trajectoire de la poussette ou la guider manuellement si le suivi automatique est désactivé.



14. Buzzer

Alerte sonore de sécurité : Il s'active immédiatement lorsque le capteur ultrason (HC-SR04) détecte un obstacle sur la trajectoire frontale.

-Offre un avertissement auditif instantané pour prévenir l'entourage ou le parent d'une collision imminente, renforçant la sécurité active du système.



15. LED

Rouge : s'allume lorsque le capteur d'orientation (MPU6050) détecte une pente dangereuse vers le bas alors que la présence du parent n'est pas détectée.

Indique visuellement un état d'urgence (risque de glissade ou de perte de contrôle autonome), déclenchant le verrouillage immédiat des moteurs du robot.

Verte : s'allume lorsque le système détecte une pente vers le haut, et dans le cas de la présence confirmée des parents.

Signale à l'utilisateur que la poussette fonctionne dans des conditions de sécurité optimales et que le suivi automatique peut s'exécuter normalement.



Bleu : s'allume brièvement ou reste fixe lorsque le récepteur radio HC-12 de la poussette reçoit un ordre manuel (appui sur les boutons gauche/droite).

Fournit un retour visuel (feedback) immédiat qui confirme que la communication sans fil entre la télécommande Arduino et le robot PIC16F887 fonctionne parfaitement.

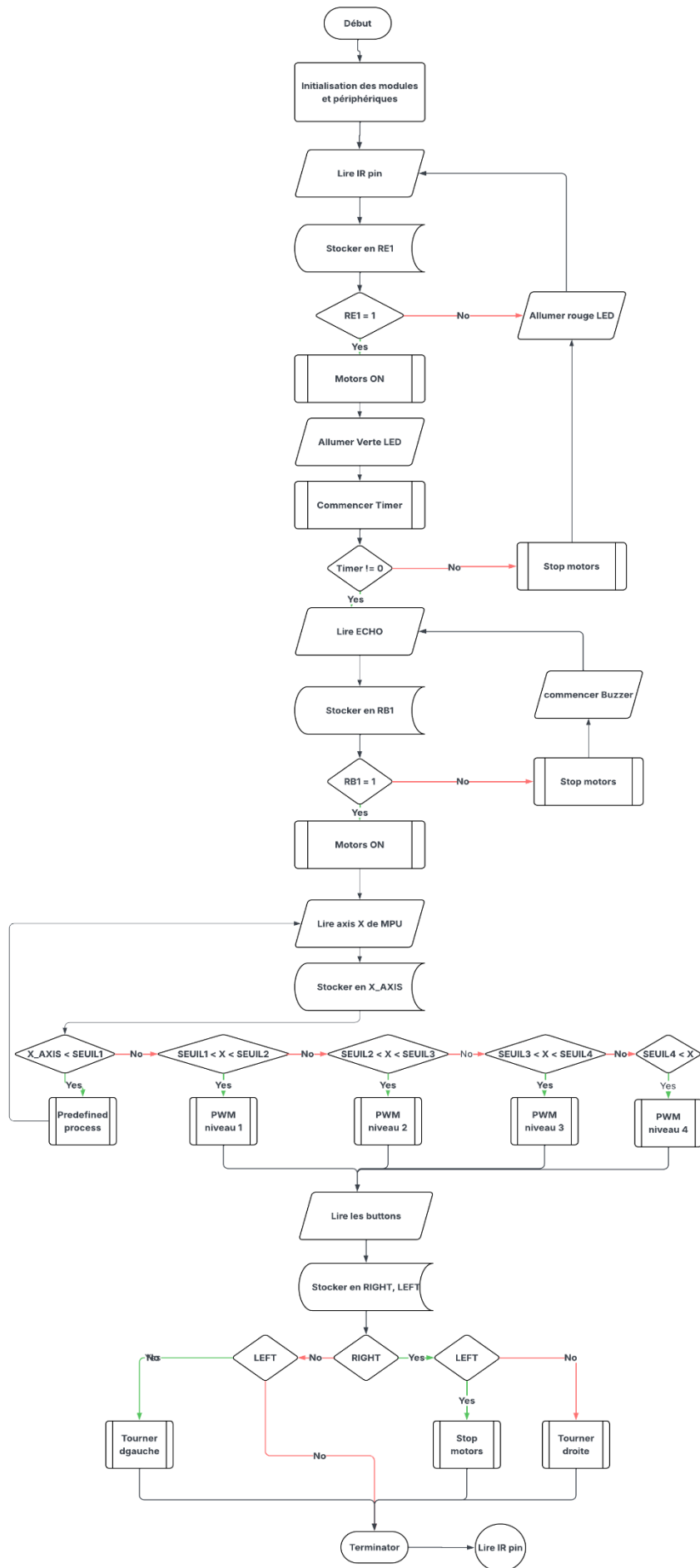
X. Liste des matériels

Pour concrétiser notre étude théorique, nous avons sélectionné des composants répondant précisément au cahier des charges fonctionnel de la poussette intelligente.

	Article	Quantité
1	Châssis avec Motors et roues (4-motors et 4-roues)	1
2	Capteur Gyroscope et Accéléromètre MPU6050	1
3	Emetteur infrarouge KY-005	1
4	Récepteur infrarouge KY-022 HX1838	3
5	Interrupteur KCD11-101	1
6	Résistances	7
7	LED	5
8	Module HC-12	2
9	Capteur Ultrason HC-SR04	1
10	Contrôleur L293D	1
11	Alimentation ajustable LM2596 3A	1
12	Batterie Li-ion 18650 3,7V	4
13	Boutons Poussoirs	2
14	Buzzer	1
15	PIC16F887	1
16	Arduino Uno	1

XI. Algorithme général du système

Le fonctionnement général peut être résumé selon les étapes suivants :



L'étude a permis de définir l'architecture matérielle et les principes de commande utilisés pour le suivi de l'utilisateur, l'évitement des obstacles et l'adaptation de la vitesse selon la pente. Ces choix constituent la base de la réalisation pratique du système qui sera développée dans le chapitre suivant.

Chapitre 3 : Simulation

La simulation permet de vérifier le bon fonctionnement du schéma électronique ainsi que le comportement des différents composants avant la réalisation pratique, offre également la possibilité de tester l'algorithme de contrôle implémenté dans le microcontrôleur PIC16F887 et de valider les différentes fonctionnalités de la poussette intelligente.

I. Présentation du logiciel Proteus ISIS



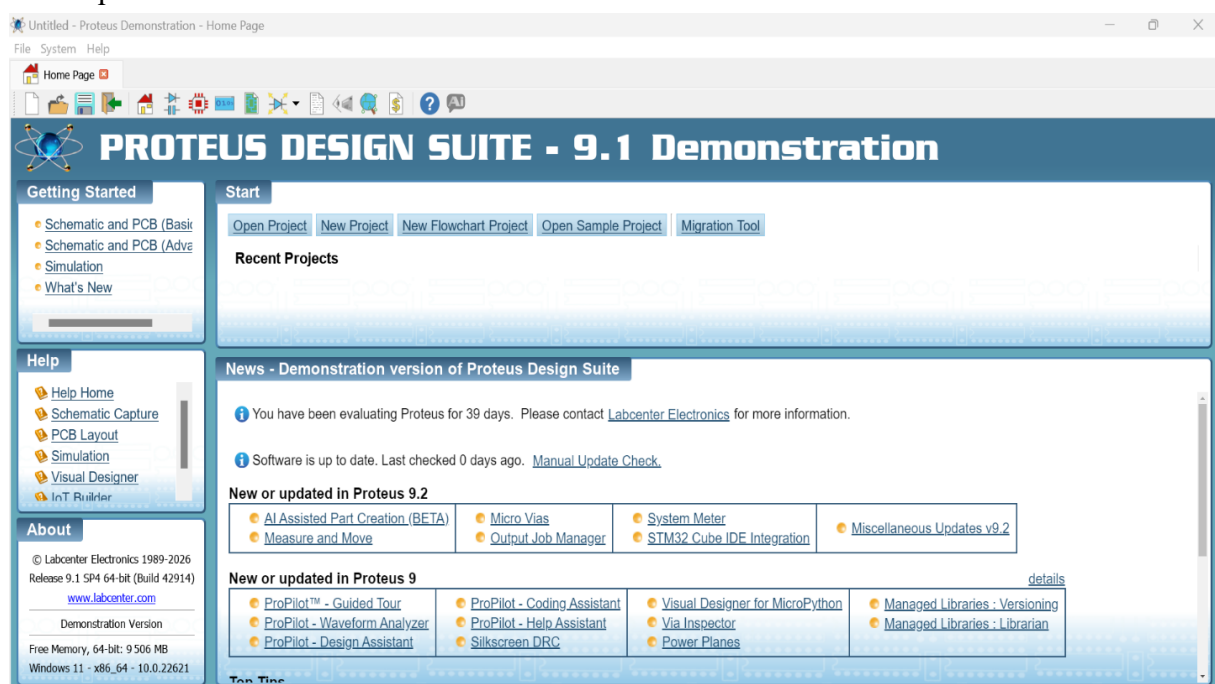
Proteus ISIS est un logiciel de conception et de simulation électronique développé par Labcenter Electronics. Il permet de réaliser des schémas électroniques, de simuler le fonctionnement des circuits analogiques et numériques ainsi que des systèmes à microcontrôleurs. Grâce à son environnement graphique intuitif, Proteus est largement utilisé dans les domaines de l'électronique, des systèmes embarqués et de l'enseignement.

1. Description d'interface du logiciel Proteus ISIS

1.1. Page d'accueil de Proteus Design Suite

Lors du lancement du logiciel Proteus Design Suite, l'utilisateur accède à une page d'accueil appelée **Home Page**. Cette interface constitue le point de départ pour la création, l'ouverture et la gestion des projets de conception et de simulation électroniques.

La page d'accueil de Proteus constitue le point d'accès principal aux différentes fonctionnalités du logiciel. Dans notre projet, elle a été utilisée pour créer un nouveau projet de simulation avant la conception du schéma électronique et la réalisation des tests virtuels du système embarqué basé sur le microcontrôleur PIC16F887.



2. Barre de menus

Située dans la partie supérieure de la fenêtre, elle contient les menus principaux :

- **File** : création, ouverture et sauvegarde des projets.
- **System** : accès aux paramètres et configurations du logiciel.
- **Help** : documentation, aide en ligne et informations sur le logiciel.

3. Barre d'outils rapide

Cette barre regroupe les commandes les plus utilisées :

- Création d'un nouveau projet.
- Ouverture d'un projet existant.
- Sauvegarde.
- Accès rapide aux différents modules de Proteus.
- Outils de gestion et de configuration.

4. Start

Cette partie centrale permet de démarrer rapidement un travail.

Ses principales options sont :

- Open Project
- New Project
- New Flowchart Project
- Open Sample Project
- Migration Tool

5. Recent Projects

Cette zone affiche les projets récemment ouverts.

Elle permet de retrouver rapidement un projet déjà utilisé sans avoir à le rechercher manuellement.

6. Getting Started

Situé à gauche de la fenêtre, ce panneau fournit des guides d'initiation :

- Schematic and PCB
- Simulation
- What's New

Ces liens permettent aux nouveaux utilisateurs de découvrir rapidement les fonctionnalités du logiciel.

7. Help

Elle constitue une source d'information importante pour apprendre à utiliser Proteus. Cette section donne accès à :

- Documentation utilisateur.
- Guides de conception de schémas.
- Aide sur la simulation.
- Tutoriels.

8. Zone d'informations et d'actualités

La partie inférieure de la page affiche :

- Les nouveautés de la version installée.
- Les mises à jour disponibles.
- Les nouvelles fonctionnalités du logiciel.

8.1. Création d'un nouveau projet sous Proteus

Lancement de l'assistant de création de projet

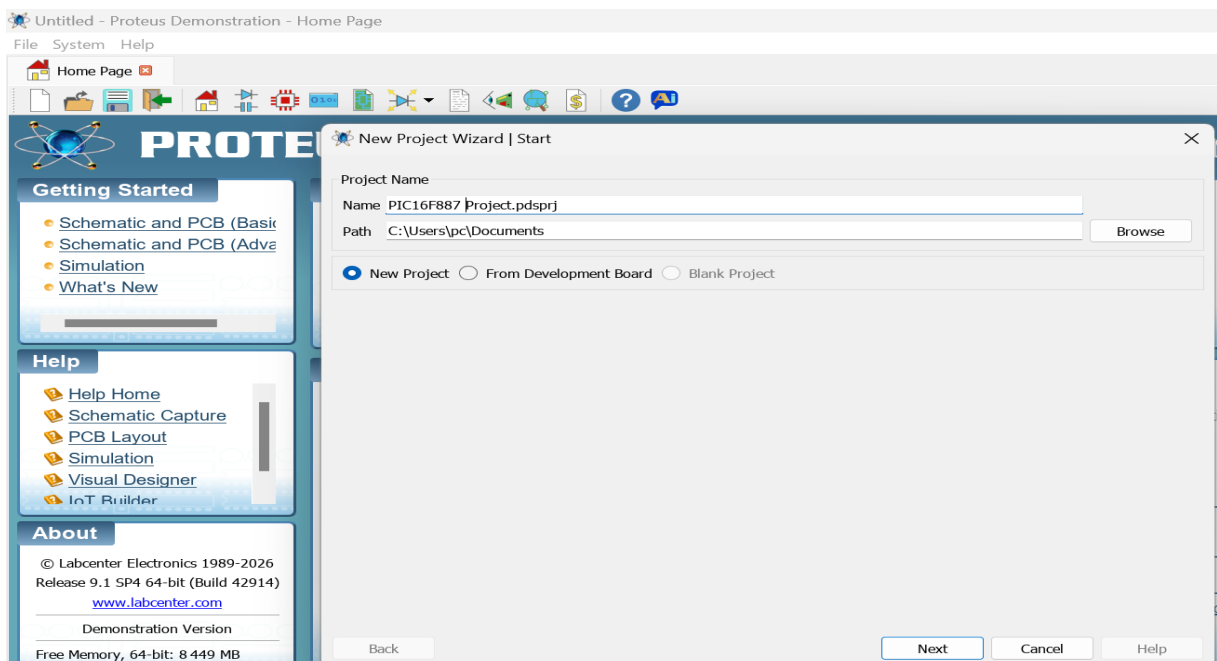
Après le démarrage du logiciel Proteus 9, l'utilisateur sélectionne l'option New Project afin de lancer l'assistant de création d'un nouveau projet.

La fenêtre New Project Wizard apparaît comme illustré dans la figure ci-dessous.

Description des champs :

- Project Name : permet de définir le nom du projet.
- Path : indique l'emplacement où seront enregistrés les fichiers du projet.
- Browse : permet de choisir un autre dossier de sauvegarde.
- New Project : crée un nouveau projet vide.
- From Développement Board : crée un projet à partir d'une carte de développement prédéfinie.
- Blank Project : crée un projet totalement vide sans configuration initiale.

Exemple



Dans notre cas :

- Nom du projet : **PIC16F887Projet**
- Emplacement : **Documents**
- Option sélectionnée : **New Project**

Une fois ces paramètres renseignés, cliquer sur le bouton **Next** pour passer à l'étape suivante.

Configuration du schéma électronique

Dans la fenêtre suivante, Proteus demande le type de projet à créer. Option recommandée Cocher Create a schematic from the selected template

Cette option permet de créer automatiquement une feuille de schéma électronique sur laquelle seront placés les composants.

Laisser le modèle par défaut ou **Default** puis cliquer sur **Next**.

Configuration du circuit imprimé (PCB)

Proteus propose ensuite la création d'un circuit imprimé. Pour une simple simulation Cocher : *Do not create a PCB layout*

Cette option est recommandée lorsque l'objectif est uniquement la simulation du montage.

Cliquer sur **Next**.

Configuration du microcontrôleur

L'assistant propose ensuite l'ajout d'un programme compilé. Pour un projet PIC16F887

Cocher :

No Firmware Project

Le fichier HEX sera ajouté ultérieurement après la compilation dans MPLAB.

Cliquer sur **Next**.

Récapitulatif du projet

Une fenêtre récapitulative affiche les paramètres sélectionnés :

- Nom du projet
- Emplacement d'enregistrement
- Type de schéma
- Configuration PCB
- Configuration Firmware

Vérifier les informations puis cliquer sur :

Finish

Ouverture de l'espace de travail

Après validation, Proteus ouvre automatiquement l'environnement de conception.

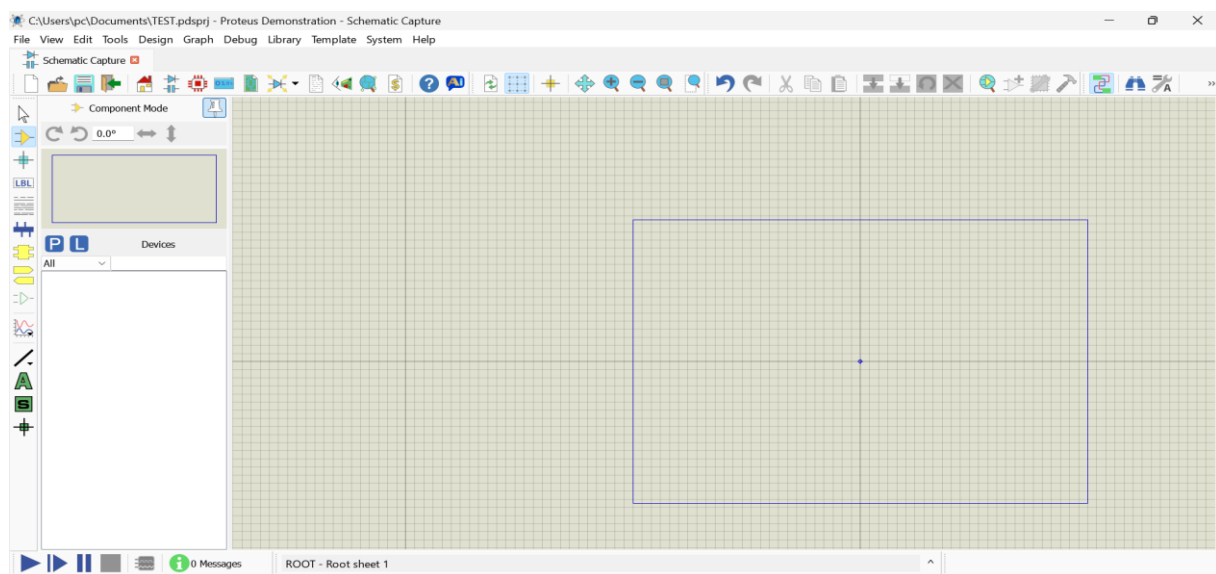
L'utilisateur peut alors :

- Ajouter des composants électroniques.
- Créer le schéma électrique.
- Configurer le microcontrôleur PIC16F887.
- Charger le fichier HEX généré par MPLAB.

Lancer la simulation du système.

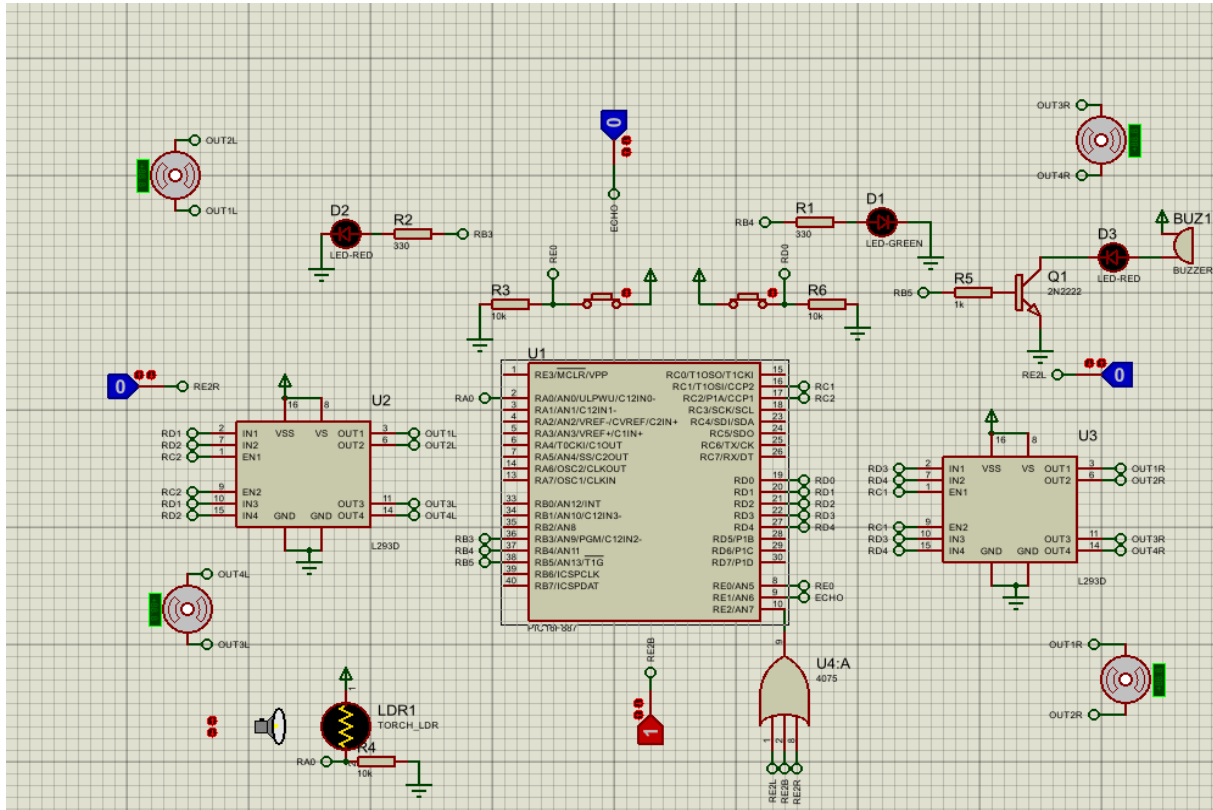
8.2. Zone de travail (Workspace)

La zone de travail constitue l'espace principal où l'utilisateur réalise le schéma électronique. Les composants y sont placés et interconnectés afin de construire le circuit à simuler.



II. Présentation du schéma

Le schéma électronique a été réalisé sous Proteus ISIS afin de simuler le fonctionnement du système avant sa réalisation matérielle. Il regroupe l'ensemble des composants nécessaires à l'implémentation du prototype et permet de vérifier le comportement du circuit dans différentes conditions de fonctionnement.



III. Description du fonctionnement général

Le système est basé sur le microcontrôleur PIC16F887 qui assure la gestion des entrées et des sorties. Les informations provenant des différents capteurs sont traitées par le microcontrôleur qui commande ensuite les actionneurs selon le programme développé sous MPLAB.

La simulation a été réalisée sous le logiciel ISIS Proteus afin de valider le fonctionnement de la poussette intelligente avant sa réalisation matérielle.

Elle comporte quatre moteurs à courant continu commandés par deux circuits L293D, des LEDs de signalisation, un buzzer, une LDR (Light Dependent Resistor) ainsi que plusieurs composants logiques utilisés pour représenter les capteurs réels. Ces derniers ont été remplacés par des composants virtuels de type « Logic State » permettant de générer des niveaux logiques 0 ou 1 afin de simuler la présence ou l'absence d'un obstacle ou du parent.

L'objectif principal de cette simulation est de vérifier le comportement global du système, notamment le déplacement de la poussette, la détection du parent, l'arrêt en présence d'un obstacle et le déclenchement des alarmes visuelles et sonores.

IV. Choix des composants de simulation

Certains composants utilisés dans le système réel ne sont pas directement disponibles dans la bibliothèque d'ISIS Proteus ou leur simulation est complexe à mettre en œuvre. Pour cette raison, des composants virtuels ont été utilisés afin de reproduire leur comportement logique.

Les capteurs infrarouges (IR) destinés à détecter la présence du parent ont été remplacés par des composants « Logic State ». Ces composants permettent de générer manuellement une valeur logique 0 ou 1 représentant respectivement l'absence ou la présence du parent.

De la même manière, le capteur ultrasonique de détection d'obstacles a été remplacé par un composant logique simulant le signal ECHO. Une valeur égale à 1 indique la présence d'un obstacle tandis qu'une valeur égale à 0 signifie qu'aucun obstacle n'est détecté.

Cette approche simplifie la simulation et permet de se concentrer sur la validation de l'algorithme de contrôle sans nécessiter l'utilisation de capteurs physiques complexes.

V. Fonctionnement global du système

Le fonctionnement du système repose sur la détection du parent et des obstacles. Lorsque au moins un capteur de présence détecte le parent, la poussette se déplace grâce aux quatre moteurs commandés par les circuits L293D. Si le parent change de direction, certains moteurs sont arrêtés afin d'effectuer une rotation vers la droite ou vers la gauche.

En revanche, lorsqu'un obstacle est détecté, le microcontrôleur arrête immédiatement tous les moteurs afin d'assurer la sécurité de l'enfant. Une alerte est alors déclenchée par le buzzer ainsi que par une LED de signalisation. Si aucun capteur ne détecte la présence du parent, la poussette s'arrête automatiquement pour éviter tout déplacement non contrôlé.

VI. Résultats de simulation

Afin de vérifier le bon fonctionnement du système, plusieurs scénarios ont été testés sous ISIS Proteus. Les résultats obtenus montrent que le comportement de la poussette est conforme aux spécifications du cahier des charges.

Cas 1 : Détection du parent

Lorsque au moins un capteur de présence détecte le parent (valeur logique 1), le microcontrôleur active les moteurs et la poussette se déplace. La LED verte D1 s'allume pour indiquer que le parent est présent.

Cas 2 : Absence du parent

Lorsque tous les capteurs de présence sont à l'état logique 0, le système considère que le parent est absent. Les moteurs sont alors arrêtés et la LED rouge D2 s'allume afin d'indiquer l'arrêt de la poussette pour des raisons de sécurité.

Cas 3 : Rotation vers la droite

Lorsque le parent change de direction vers la droite, les moteurs situés du côté droit sont arrêtés tandis que les moteurs du côté gauche continuent de fonctionner. La poussette effectue ainsi une rotation vers la droite.

Cas 4 : Rotation vers la gauche

Lorsque le parent se déplace vers la gauche, les moteurs du côté gauche sont arrêtés et ceux du côté droit restent actifs, permettant à la poussette de tourner vers la gauche.

Cas 5 : Détection d'un obstacle

Lorsqu'un obstacle est détecté, le signal ECHO passe à l'état logique 1. Le microcontrôleur arrête immédiatement les quatre moteurs afin d'assurer la sécurité de l'enfant. Le buzzer est activé et la LED D3 s'allume pour signaler la présence de l'obstacle.

Cas 6 : Fonctionnement du capteur LDR

Le capteur LDR a été testé en faisant varier l'intensité lumineuse de la source TORCH. Les variations de luminosité sont correctement détectées par le microcontrôleur. Selon le programme implémenté, ces informations peuvent être utilisées pour adapter certains paramètres du système, tels que la vitesse des moteurs.

Les résultats obtenus montrent que le système répond correctement aux différentes situations testées. La simulation a ainsi permis de valider le schéma électronique, l'algorithme de contrôle et le comportement global de la poussette intelligente avant sa réalisation pratique.

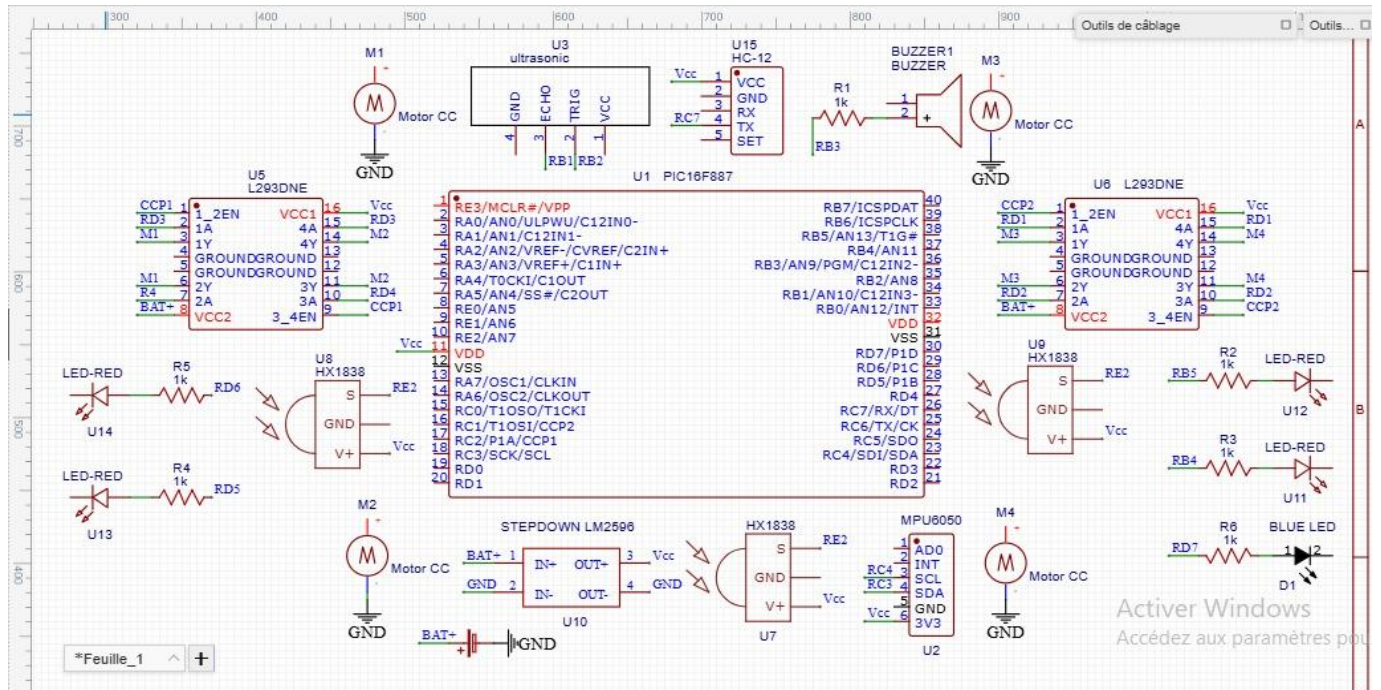
Chapitre 4 : Réalisation et Test de validation

I. Introduction

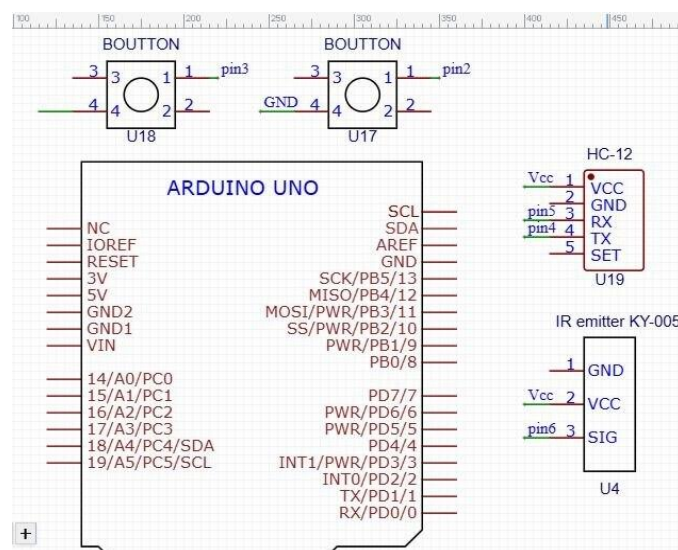
Après la validation de la simulation, ce dernier chapitre aura pour objectif la mise en évidence de résultat de la partie pratique obtenue.

II. Schéma électronique du montage

Poussette intelligente



Télécommande



III. Développement du logiciel

1. MPLAB X IDE

MPLAB X IDE est un environnement de développement officiel de Microchip utilisé pour programmer les microcontrôleurs PIC.

Il permet de :

- Créer et gérer des projets embarqués
- Écrire du code en Assembleur ou en C
- Compiler et simuler les programmes
- Programmer le microcontrôleur

1.1. Création d'un projet PIC16F887

Dans MPLAB X IDE v5.35, la création d'un projet pour le PIC16F887 suit ces étapes :

- Sélection du microcontrôleur : PIC16F887
- Choix de l'outil de programmation : PICKit 3 (ou autre programmeur)
- Choix du compilateur/assembleur (MPASM ou XC8 avec ASM intégré)
- Création du projet et organisation des fichiers source

Le projet génère une structure contenant :

- Fichiers source (.asm)
- Fichiers de configuration
- Dossier de compilation

1.2. Programmation en Assembleur (ASM)

Le langage Assembleur est un langage bas niveau proche du matériel.

Dans le PIC16F887 :

- Chaque instruction agit directement sur les registres
- Utilisation des ports (PORTA, PORTB, etc.)
- Configuration des registres SFR (Special Function Registers)

Exemples d'opérations :

- Configuration des ports en entrée/sortie
- Gestion des temporisations avec les timers
- Contrôle des capteurs et moteurs du robot

1.3. Génération du fichier HEX

Après écriture du programme :

- Le code est assemblé (build)
- Le compilateur transforme le code ASM en code machineA
- Génération du fichier HEX

Ce fichier contient le programme final qui sera chargé dans le microcontrôleur.

Programmation du microcontrôleur

Le fichier HEX est transféré vers le PIC16F887 via :

- PICKit 3
- Connexion ICSP (In-Circuit Serial Programming)

Le microcontrôleur exécute ensuite directement le programme du robot.

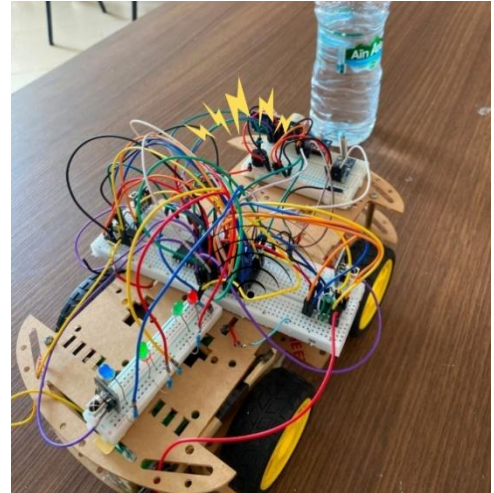
IV. Tests et Validation

1. Test du Système d'évitement d'obstacle

Afin de vérifier le bon fonctionnement du système d'évitement d'obstacles, plusieurs essais ont été réalisés en plaçant différents obstacles devant la poussette autonome. Le capteur à ultrasons assure la mesure continue de la distance entre la poussette et l'obstacle.

Lors des essais, dès qu'un obstacle est détecté à une distance inférieure au seuil de sécurité défini, le microcontrôleur interrompt immédiatement le déplacement de la poussette afin d'éviter toute collision. Simultanément, le buzzer est activé pour produire une alerte sonore signalant la présence de l'obstacle.

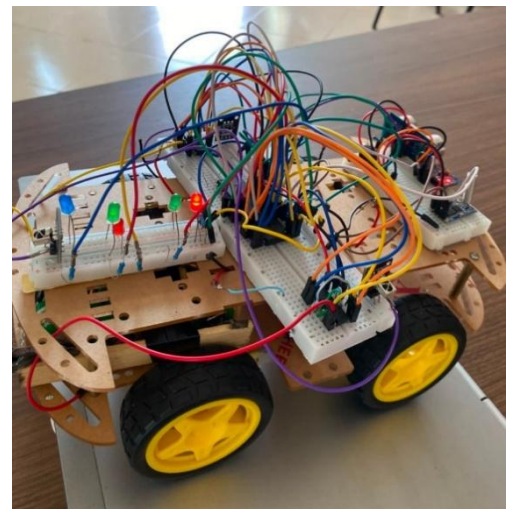
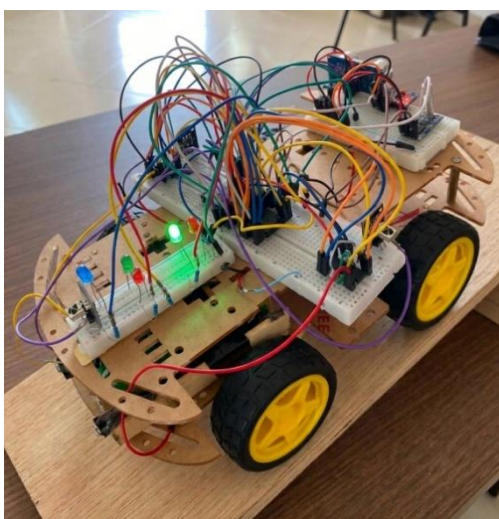
Les différents tests réalisés montrent que le système réagit correctement à la présence d'un obstacle. L'arrêt est effectué de manière instantanée et le signal sonore permet d'avertir efficacement l'utilisateur. Les résultats obtenus confirment ainsi le bon fonctionnement du système d'évitement d'obstacles.



2. Test du Système de Détection de Pente

Le fonctionnement du système de détection de pente a été validé en plaçant la poussette sur différentes inclinaisons du terrain. Le capteur MPU6050 mesure en temps réel l'angle d'inclinaison et transmet les informations au microcontrôleur.

Lorsque la poussette est placée sur une pente descendante, la LED rouge est activée pour signaler la descente, tandis que la vitesse diminue progressivement en fonction de l'inclinaison jusqu'à l'arrêt complet des moteurs en cas de pente très raide.



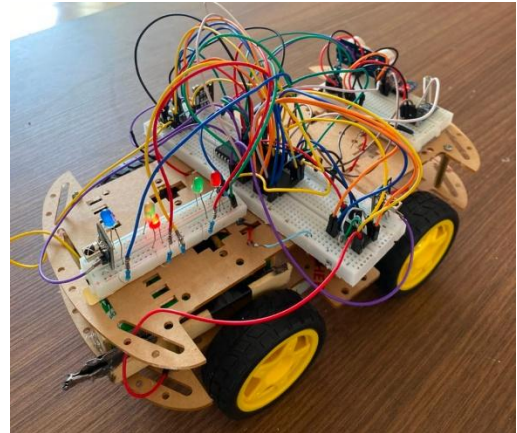
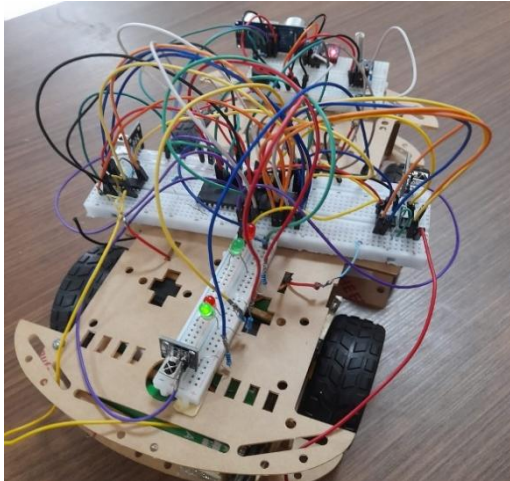
Les essais réalisés montrent que lorsque la poussette se trouve sur une pente montante, la LED verte s'allume pour indiquer que le système a correctement détecté cette situation, tandis que la vitesse augmente progressivement jusqu'à atteindre sa valeur maximale en fonction du degré de l'inclinaison..

Les résultats expérimentaux démontrent que le capteur détecte correctement les variations d'inclinaison et que le système réagit conformément aux conditions définies lors de la conception.

3. Test de système de détection de présence de parents

Le système de suivi du parent a été testé à l'aide des capteurs infrarouges installés sur la poussette.

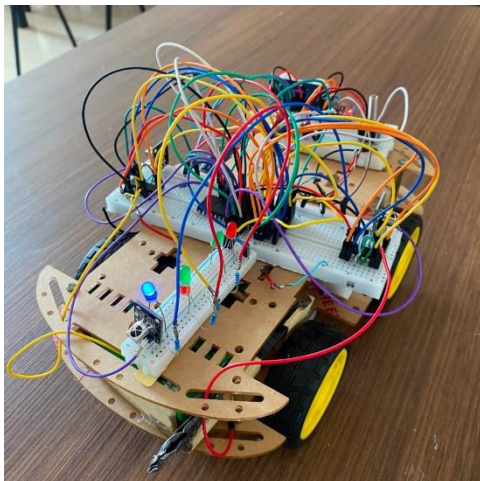
Lorsque le parent n'est plus détecté, la LED rouge s'allume afin de signaler son absence. Cette indication confirme que le système identifie correctement la perte de détection et applique le comportement de sécurité prévu.



En revanche, lorsque le parent est détecté par les capteurs, la LED verte s'allume, indiquant que la présence de l'utilisateur est correctement reconnue par le système. Dans cette situation, la poussette est autorisée à poursuivre son fonctionnement normal et à exécuter les commandes reçues.

Les différents essais effectués montrent que le système distingue correctement les deux états de fonctionnement (présence et absence du parent), ce qui garantit une meilleure sécurité lors de l'utilisation de la poussette.

4. Test du Système de Commande de moteur par Télécommande



Les essais effectués montrent que la communication entre l'émetteur et le récepteur est fiable et réactive. L'activation instantanée de la LED bleue lors de la réception des données confirme le bon fonctionnement du protocole de transmission.

Le système de télécommande est donc validé, car il permet une communication efficace entre l'utilisateur et la poussette autonome, assurant ainsi la transmission des ordres de commande en temps réel.

V. Analyse de performance

Les différents essais réalisés ont permis de valider l'ensemble des principales fonctionnalités de la poussette autonome. Le système de détection d'obstacles a démontré une réaction rapide en arrêtant immédiatement les moteurs et en déclenchant une alerte sonore dès la détection d'un obstacle. Le système de détection de pente a correctement identifié les montées et les descentes à l'aide du capteur MPU6050, ce qui confirme la fiabilité de la mesure de l'inclinaison. Enfin,

le système de détection du parent a assuré une reconnaissance fiable de sa présence grâce aux capteurs infrarouges, garantissant ainsi le respect des conditions de sécurité prévues.

Dans l'ensemble, les résultats expérimentaux montrent que les différents modules matériels et logiciels fonctionnent de manière cohérente et répondent aux objectifs fixés au début du projet. La communication entre les capteurs, le microcontrôleur et les actionneurs s'est révélée stable durant les essais, permettant à la poussette de réagir correctement aux différentes situations rencontrées. Ces tests confirment ainsi la faisabilité et le bon fonctionnement du système développé.

VI. Limites du système et Difficultés rencontrés :

1. Limites du système

Le prototype de la poussette intelligente hybride (Suivi IR + Commande Radio) a validé les principes de fonctionnement de base. Cependant, l'évaluation en conditions réelles a mis en évidence plusieurs limites techniques :

- **Dépendance à la ligne de visée (Line-of-Sight) :** Le suivi automatique repose sur l'alignement optique entre l'émetteur porté par l'utilisateur et les trois récepteurs **KY-022**. Si l'utilisateur tourne brusquement ou si un obstacle opaque (un vêtement, un sac) masque l'émetteur, la poussette perd le signal et se met en sécurité (arrêt instantané).
- **Sensibilité aux conditions météorologiques :**
 - Rayonnement solaire (Éblouissement) :** Le spectre de la lumière directe du soleil contient une forte composante infrarouge. En extérieur et par temps très ensoleillé, Les récepteurs subissent un phénomène de saturation, ce qui réduit drastiquement la portée utile du suivi automatique.
 - Pluie et humidité :** Les précipitations perturbent fortement le système. L'eau de pluie peut fausser les mesures du capteur ultrason **HC-SR04** en diffractant les ondes acoustiques. De plus, le prototype actuel ne possède pas de châssis étanche (indice de protection IP adéquat), ce qui expose les circuits (PIC16F887, modules) à des risques de court-circuit.

2. Difficultés rencontrées

Le développement de ce système embarqué dès le début jusqu'à la fin a présenté plusieurs défis techniques majeurs qu'il a fallu surmonter :

- **Complexité de programmation bas niveau du PIC16F887 :** Le développement sur le PIC16F887 avec le compilateur a nécessité une configuration manuelle et rigoureuse de chaque registre interne (TRIS, ANSEL, INTCON). La gestion du timing précis pour le capteur ultrason via le Timer1 et la configuration des interruptions ont représenté une difficulté logicielle importante par rapport à la simplicité de l'Arduino utilisé pour la télécommande.
Cela nécessite une analyse rigoureuse de la datasheet du composant et l'utilisation de la simulation pas à pas sur MPLAB X pour valider le comportement des registres avant l'injection du code.
- **Conflits et coexistence de protocoles de communication multiples :**
Le cœur du problème résidait dans la gestion simultanée de l'**I2C** (pour le MPU6050), de l'**UART** (pour le HC-12) et des signaux temporels de l'**Infrarouge** (KY-022). Les fonctions bloquantes classiques empêchaient le PIC de traiter les données en temps réel, ce qui causait des freezes du robot ou des ordres manuels manqués.
- **Synchronisation et croisement UART (Module HC-12) :**

La mise en place de la communication série bidirectionnelle entre l'Arduino de la télécommande et le PIC16F887 a posé des difficultés de communication initiales (données corrompues ou non reçues).

Solution : Une vérification rigoureuse du câblage (croisement impératif de la broche TX vers la broche RX) combinée à une configuration identique de la vitesse de transmission (Baudrate à 9600 bps) a permis de stabiliser la liaison radio.

- **Algorithme de tri des signaux IR :**

Avec trois capteurs KY-022 espacés à l'avant, il a été difficile au début d'interpréter correctement les données reçues pour savoir si l'utilisateur était plutôt à gauche ou au centre, à cause des reflets de la lumière sur les murs.

Conclusion général

Ce projet a permis de concevoir et de réaliser une poussette autonome intelligente capable d'assister les parents lors de leurs déplacements tout en renforçant la sécurité de l'enfant. L'objectif principal était de développer un système embarqué capable de suivre son utilisateur, de détecter les obstacles présents sur son parcours et d'adapter son comportement en fonction de l'environnement.

Pour atteindre cet objectif, plusieurs technologies ont été intégrées autour du microcontrôleur PIC16F887. Les capteurs infrarouges assurent la détection de la présence du parent, le capteur à ultrasons garantit l'évitement des obstacles, tandis que le capteur MPU6050 permet d'identifier les variations de pente afin d'adapter automatiquement la vitesse de déplacement. La communication entre la télécommande et la poussette est réalisée grâce aux modules HC-12, permettant à l'utilisateur de transmettre les commandes de direction à distance. Enfin, un système d'alerte composé de LED et d'un buzzer améliore la sécurité en informant immédiatement l'utilisateur des différentes situations détectées.

Les différents essais réalisés ont confirmé le bon fonctionnement des principales fonctionnalités développées. Les résultats obtenus montrent que la poussette est capable de détecter efficacement la présence ou l'absence du parent, d'identifier les obstacles situés sur son trajet, d'émettre les alertes correspondantes et de reconnaître les changements d'inclinaison du terrain. L'ensemble des modules matériels et logiciels fonctionne de manière cohérente, démontrant ainsi la faisabilité de la solution proposée.

Bien que le prototype réponde aux objectifs fixés, plusieurs améliorations peuvent être envisagées afin d'augmenter les performances et les fonctionnalités du système. Parmi celles-ci figurent l'intégration d'une caméra associée à des techniques de vision artificielle pour un suivi plus précis du parent, l'utilisation de capteurs supplémentaires pour améliorer la détection de l'environnement, l'ajout d'un système de localisation GPS ainsi que l'emploi d'un microcontrôleur plus performant afin de gérer des algorithmes plus complexes.

En conclusion, ce projet constitue une application concrète des systèmes embarqués, de l'électronique et de l'automatique dans le domaine de la mobilité intelligente. Il met en évidence l'intérêt de combiner plusieurs capteurs et actionneurs afin de concevoir un système autonome fiable, capable d'améliorer le confort et la sécurité des utilisateurs, tout en offrant une base solide pour le développement de futures solutions de poussettes intelligentes.

Annexes :

Code source utilisé dans la simulation en langage assembleur

```
;=====
; PROJET : POUSSETTE INTELLIGENTE
;=====

LIST p=16f887
INCLUDE "p16f887.inc"

; ---- CONFIGURATION DES FUSIBLES ----

__CONFIG __CONFIG1, _LVP_OFF & _FCMEN_OFF & _IESO_OFF & _BOR_OFF &
_CPD_OFF & _CP_OFF & _MCLRE_ON & _PWRTE_ON & _WDT_OFF & _HS_OSC
__CONFIG __CONFIG2, _WRT_OFF & _BOR21V

; ---- VARIABLES ----

CBLOCK 0x20
    COMPTEUR_5S      ; Compteur pour les 5 secondes (50 boucles de 100ms)
    VAR1, VAR2, VAR3  ; Variables pour les boucles de délai
    REG_ADC           ; Valeur de la LDR (0-255)
    FLAG_ETAT         ; Bit 0: 1=Poussette en marche, 0=Arrêtée
ENDC

ORG 0x0000
    GOTO INITIALISATION

;=====
; SOUS-ROUTINE : DELAI DE 100 MILLISECONDES
;=====

DELA1_100MS:
    MOVLW .2
    MOVWF VAR3
```


BOUCLE_EXT:

MOVLW .130

MOVWF VAR2

BOUCLE_MID:

MOVLW .255

MOVWF VAR1

BOUCLE_INT:

DECFSZ VAR1, F

GOTO BOUCLE_INT

DECFSZ VAR2, F

GOTO BOUCLE_MID

DECFSZ VAR3, F

GOTO BOUCLE_EXT

RETURN

;

; SOUS-ROUTINE : LECTURE DU CAPTEUR LDR

;

LECTURE_LDR:

BANKSEL ADCON0

BSF ADCON0, GO ; Lancer la conversion Analogique/Numérique

ATTENTE_ADC:

BTFSC ADCON0, GO

GOTO ATTENTE_ADC

MOVF ADRESH, W

MOVWF REG_ADC ; Stocker le résultat (0 à 255)

RETURN

```
;=====
; SOUS-ROUTINE : ARRÊT TOTAL DES MOTEURS
;=====
```

ROUTINE_ARRET:

```
    CLRF CCPR1L      ; PWM Gauche à 0%
    CLRF CCPR2L      ; PWM Droite à 0%
    MOVLW B'00000000'
    MOVWF PORTD      ; Couper complètement les directions (L293D)
    BCF PORTB, 7      ; Eteindre LED Vitesse Verte
    BCF PORTB, 6      ; Eteindre LED Vitesse Rouge
    RETURN
```

```
;=====
; INITIALISATION DU MICROCONTRÔLEUR
;=====
```

INITIALISATION:

```
    BANKSEL TRISA
    MOVLW B'00000001' ; RA0 (AN0) en entrée (LDR)
    MOVWF TRISA
    CLRF TRISB        ; PORTB en sortie (LEDs et Buzzer)
    CLRF TRISC        ; PORTC en sortie (PWM sur RC1, RC2)
    MOVLW B'00000001' ; RD0 entrée (Btn Droit), RD1-RD4 sorties (Moteurs)
    MOVWF TRISD
    MOVLW B'00000111' ; RE0 (Btn Gauche), RE1 (ECHO), RE2 (IR) en entrées
    MOVWF TRISE

    BANKSEL ANSEL
    MOVLW B'00000001' ; AN0 Analogique
    MOVWF ANSEL
    CLRF ANSELH
```

```
BANKSEL ADCON1
CLRF ADCON1
BANKSEL ADCON0
MOVLW B'10000001'
MOVWF ADCON0
```

; Configuration PWM Matériel

```
BANKSEL PR2
MOVLW .124
MOVWF PR2
BANKSEL T2CON
MOVLW B'00000110'
MOVWF T2CON
BANKSEL CCP1CON
MOVLW B'00001100'
MOVWF CCP1CON
MOVLW B'00001100'
MOVWF CCP2CON
```

```
BANKSEL PORTA
CLRF PORTC
CLRF PORTD
CLRF PORTB
CLRF FLAG_ETAT ; Initialiser l'état de la poussette à l'arrêt
```

```
;=====
; BOUCLE PRINCIPALE
;=====
```

BOUCLE_PRINCIPALE:

; 1. Sécurité Obstacle (Ultrason sur RE1)

BTFSC PORTE, 1

GOTO MODE_OBSTACLE

; 2. Vérification Présence Parent (IR sur RE2)

BTFSC PORTE, 2

GOTO MARCHE_NORMALE

; 3. Parent Absent !

; On vérifie si la poussette était en marche juste avant

BTFSC FLAG_ETAT, 0

GOTO DEBUT_COMPTE_5S ; Elle était en marche, lancer le compte à rebours

; Sinon, elle était déjà à l'arrêt, on maintient l'état

GOTO ETAT_ARRET

MARCHE_NORMALE:

BSF FLAG_ETAT, 0 ; Mémoriser que la poussette est en mouvement

BCF PORTB, 3 ; Eteindre LED Rouge Parent

BSF PORTB, 4 ; Allumer LED Verte Parent

CALL GESTION_MOTEURS ; Avancer normalement

GOTO BOUCLE_PRINCIPALE

DEBUT_COMPTE_5S:

; Le parent vient de disparaître : on maintient la LED Verte allumée
; pendant les 5 secondes car la poussette roule encore.

BCF PORTB, 3 ; Assurer que la LED Rouge est éteinte

BSF PORTB, 4 ; Garder la LED Verte allumée

MOVLW .50 ; 50 x 100ms = 5 secondes

MOVWF COMPTEUR_5S

BOUCLE_5S:

CALL GESTION_MOTEURS ; Continuer à faire rouler la poussette

CALL DELAI_100MS ; Attendre 100ms

; Vérifier si le parent est revenu

BTFSC PORTE, 2

GOTO MARCHE_NORMALE ; Il est revenu ! Annuler le compte et continuer

; Vérifier s'il y a un obstacle pendant le compte à rebours

BTFSC PORTE, 1

GOTO MODE_OBSTACLE

DECFSZ COMPTEUR_5S, F

GOTO BOUCLE_5S

; Fin des 5 secondes sans retour du parent -> Arrêt effectif

GOTO ETAT_ARRET

ETAT_ARRET:

BCF FLAG_ETAT, 0 ; Mémoriser que la poussette est arrêtée

BSF PORTB, 3 ; Allumer LED Rouge (Moteurs arrêtés, parent absent)

BCF PORTB, 4 ; Eteindre LED Verte

CALL ROUTINE_ARRET ; Couper l'alimentation des moteurs

GOTO BOUCLE_PRINCIPALE

MODE_OBSTACLE:

CALL ROUTINE_ARRET ; Couper les moteurs immédiatement

BSF PORTB, 5 ; Allumer Buzzer

; Délai 1 seconde (10 x 100ms) pour le buzzer

MOVLW .10

MOVWF COMPTEUR_5S

BOUCLE_BUZZER:

CALL DELAI_100MS

DECFSZ COMPTEUR_5S, F

GOTO BOUCLE_BUZZER

BCF PORTB, 5 ; Eteindre Buzzer

GOTO BOUCLE_PRINCIPALE

; =====

; GESTION MOTEURS ET PWM

; =====

GESTION_MOTEURS:

; --- A. CONTRÔLE DE LA DIRECTION ---

BTFSC PORTE, 0 ; Le bouton Gauche (RE0) est-il pressé (1) ?

GOTO BTN_GAUCHE_PRESSE

BTFSC PORTD, 0 ; Le bouton Droite (RD0) est-il pressé (1) ?

GOTO BTN_DROITE_SEUL

; MARCHÉ AVANT LIGNE DROITE (Aucun bouton pressé)

MOVLW B'00001010' ; RD1=1, RD2=0 (Gauche Avant) | RD3=1, RD4=0 (Droite Avant)

```
MOVWF PORTD
GOTO ADAPTE_VITESSE
```

BTN_GAUCHE_PRESSE:

```
BTFSC PORTD, 0    ; Le bouton droit est-il AUSSI pressé ?
GOTO ARRET_CONFLIT ; Oui -> Conflit -> Arrêt de sécurité

; TOURNER À GAUCHE (Moteur Gauche à l'arrêt, Moteur Droit avance)
MOVLW B'00001000' ; RD1=0, RD2=0 (Gauche STOP) | RD3=1, RD4=0 (Droite
Avant)
MOVWF PORTD
GOTO ADAPTE_VITESSE
```

BTN_DROITE_SEUL:

```
; TOURNER À DROITE (Moteur Gauche avance, Moteur Droit à l'arrêt)
MOVLW B'00000010' ; RD1=1, RD2=0 (Gauche Avant) | RD3=0, RD4=0 (Droite
STOP)
MOVWF PORTD
GOTO ADAPTE_VITESSE
```

ARRET_CONFLIT:

```
CALL ROUTINE_ARRET
RETURN
```

ADAPTE_VITESSE:

```
; --- B. LECTURE LDR ET SÉLECTION PWM ---
CALL LECTURE_LDR

MOVLW .240
SUBWF REG_ADC, W
BTFSC STATUS, C
```

GOTO VIT_7

MOVLW .200

SUBWF REG_ADC, W

BTFSC STATUS, C

GOTO VIT_6

MOVLW .160

SUBWF REG_ADC, W

BTFSC STATUS, C

GOTO VIT_5

MOVLW .130

SUBWF REG_ADC, W

BTFSC STATUS, C

GOTO VIT_4 ; Zone médiane

MOVLW .90

SUBWF REG_ADC, W

BTFSC STATUS, C

GOTO VIT_3

MOVLW .50

SUBWF REG_ADC, W

BTFSC STATUS, C

GOTO VIT_2

GOTO VIT_1 ; Si inférieur à 50

VIT_7:

MOVLW .124

GOTO APPLIQUE_PWM

VIT_6:

MOVLW .110

GOTO APPLIQUE_PWM

VIT_5:

MOVLW .90

GOTO APPLIQUE_PWM

VIT_4:

MOVLW .75 ; Vitesse Médiane (Croisière)

GOTO APPLIQUE_PWM

VIT_3:

MOVLW .60

GOTO APPLIQUE_PWM

VIT_2:

MOVLW .40

GOTO APPLIQUE_PWM

VIT_1:

MOVLW .20 ; Vitesse très lente

GOTO APPLIQUE_PWM

APPLIQUE_PWM:

MOVWF CCPR1L ; Envoi au moteur Gauche (RC2)

MOVWF CCPR2L ; Envoi au moteur Droit (RC1)

; --- C. GESTION DES LEDS DE VITESSE ---

MOVLW .110

SUBWF REG_ADC, W

BTFSS STATUS, C ; Si ADC < 110

GOTO LDR_BASSE

```
MOVLW .146
```

```
SUBWF REG_ADC, W
```

```
BTFSS STATUS, C ; Si ADC entre 110 et 145 (Point mort)
```

```
GOTO LDR_MEDIANE
```

```
; Si ADC >= 146
```

```
GOTO LDR_HAUTE
```

```
LDR_BASSE:
```

```
BSF PORTB, 6 ; Allumer LED Rouge Vitesse (Ralentissement)
```

```
BCF PORTB, 7 ; Eteindre LED Verte
```

```
RETURN
```

```
LDR_MEDIANE:
```

```
BCF PORTB, 6 ; Eteindre LED Rouge
```

```
BCF PORTB, 7 ; Eteindre LED Verte
```

```
RETURN
```

```
LDR_HAUTE:
```

```
BCF PORTB, 6 ; Eteindre LED Rouge
```

```
BSF PORTB, 7 ; Allumer LED Verte Vitesse (Accélération)
```

```
RETURN
```

```
END.
```